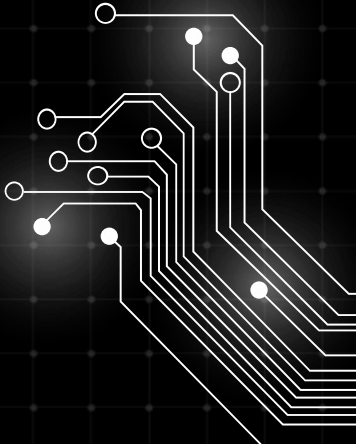
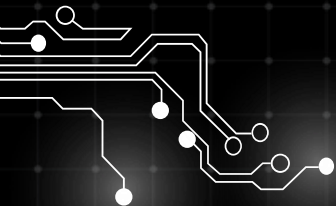


Multi-Agent Pathfinding (MAPF)

Kevin Cruz, Chahit Jain, Andrea Turek, Jessica Xu

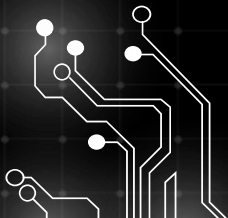
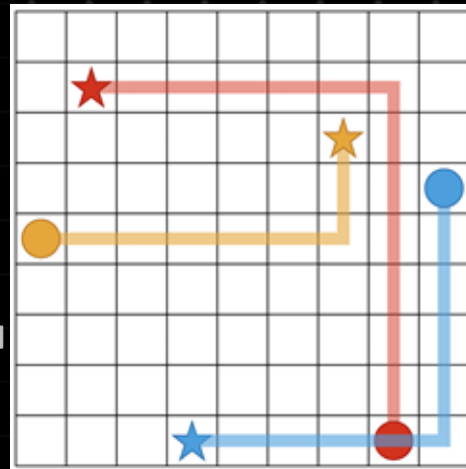
Introduction to MAPF

- Multiple agents moving on a graph
- Must avoid collisions
- Optimize:
 - Sum-of-Costs → sum of steps taken by all agents
 - Makespan → total time until last agent reaches destination

Need: Collision-free paths

Want: Fast and efficient solutions

This is difficult because complexity grows quickly with more agents



Approaches

Decoupled

+

Coupled

=

Hybrid

Plans for each robot separately

- Fast & scalable
- Not complete
- Not optimal

Plans for all robots at the same time in the joint space

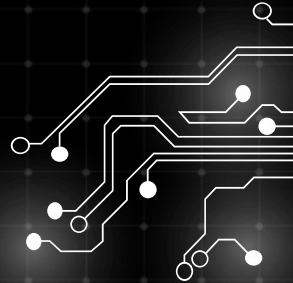
- Complete
- Optimal
- Computationally expensive
- Very slow
- Scales exponentially

Combines **both** ideas, many variations of this

- Plan robots separately, deal with conflicts jointly
- Leverage speed of decoupled & completeness/optimalty of coupled



Foundational Research



Hybrid Methods Reviewed

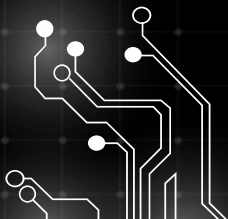
Method	What it Does	Works Best When...
CBS	Plans paths independently, resolves conflicts using a constraint tree	Few conflicts, structured environments (e.g., corridors)
M*	Starts planning independently and only couples agents when conflicts occur	Sparse interactions, conflicts are rare/local
SIPP	Plans in space-time using safe time intervals to avoid collisions	Dynamic environments, timing constraints matter
Incremental Coordination	Plans agents sequentially, adjusting plans as conflicts arise	Medium-density environments, moderate conflicts
ICBS / ICobot-style	Improved CBS with heuristics, prioritization, and conflict handling strategies	Larger problems where basic CBS struggles
SAT-Based Solver	Encodes MAPF as a SAT problem and solves globally	Dense environments, highly constrained problems



Key Insight

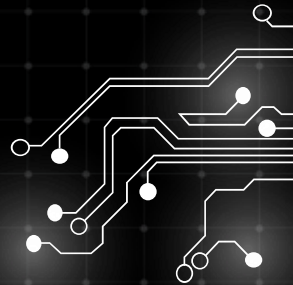
- All methods are hybrid... but in different ways
- Performance depends heavily on the environment
- Key factors:
 - Conflict density (few vs many)
 - Structure (open vs narrow spaces)

No single algorithm dominates in all circumstances



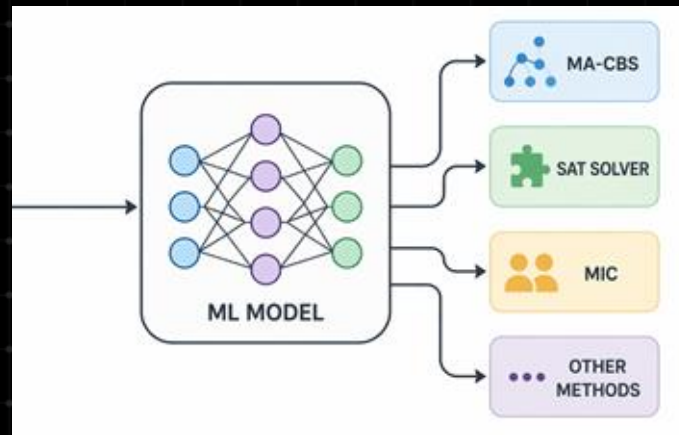


Our Research Direction



Our Research Direction

- Different algorithms perform best in different settings
- We want to automatically choose the right one
- **Goal:** build an adaptive selection framework



Select the best MAPF algorithm based on the environment

Prior Work

ML Algorithm Selection

- Uses machine learning to select MAPF solvers
- Approach 1) Based on features like:
 - # of rows and columns
 - # of obstacles
 - ratio of blocked cells
 - # of agents
 - branching factor of A*
 - average, max, min distance between sources and targets
- Approach 2) Image + CNN
 - Obstacles **white**, free space **black**
 - Start positions **green**, goal positions **red**
 - Let a CNN “see” the problem and choose the algorithm

The ML selector solves **more problems** and **faster** than any single algorithm.

Algorithm Selection for Optimal Multi-Agent Pathfinding

Omri Kaduri,¹ Eli Boyarski,¹ Roni Stern^{1,2}

¹Ben Gurion University of the Negev, SISE Dept., Be'er Sheva, Israel

²Palo Alto Research Center (PARC), SSL, Palo Alto, USA

kaduro@post.bgu.ac.il, eli@boyar.ski, roni.stern@gmail.com

Model	Accuracy	Coverage	Total RT
A* +ODwID	0.00	0.73	12,459
EPEA*	0.12	0.79	9,871
ICTS	0.10	0.78	10,570
CBS	0.00	0.59	17,447
MA-CBS	0.26	0.53	19,952
CBS-H	0.50	0.83	8,016
Random	0.16	0.71	13,215
CNN Rg.	0.51	0.88	6,210
XGBoost Rg.	0.51	0.82	8,410
CNN CL	0.55	0.91	5,122
XGBoost CL	0.66	0.93	4,307
Oracle	1.00	100.00	1,649

Figure 2: Results for all models across all the test set.

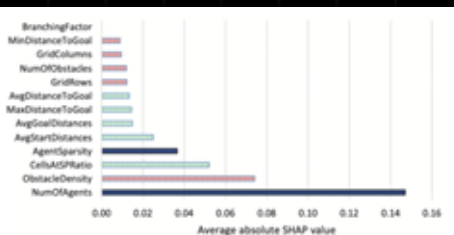


Figure 4: Feature importance calculated using SHAP values.

Our Improvement

1) Better Problem Understanding

- Learn how to characterize MAPF environments
- Identifying and encoding features:
 - bottlenecks
 - conflict density
 - narrow passages
- Use this to guide algorithm selection

2) MAPF Advanced Planning

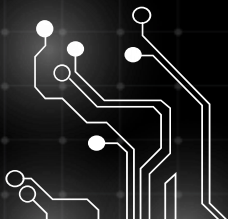
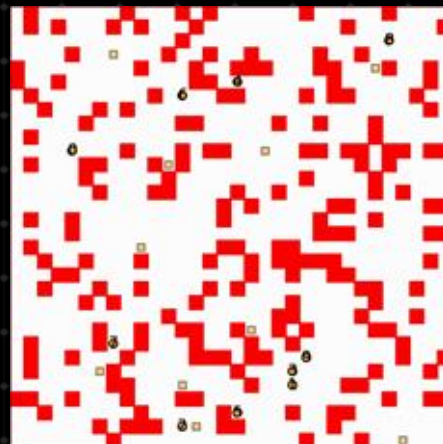
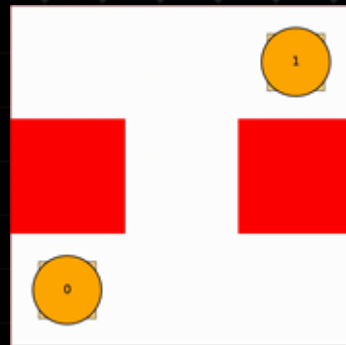
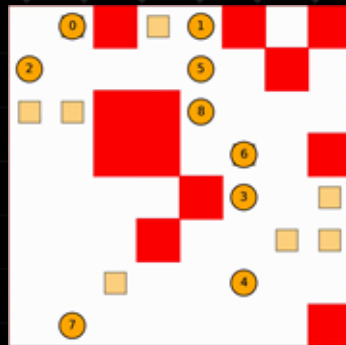
- Use MAPF to guide more complex planners
- Example:
 - kinodynamic planning (motion, velocity constraints)
- Key challenge:
 - how MAPF incorporates real-world constraints

3) Better Decoupling

- Avoid planning for all agents at once
- Find:
 - good partitions
 - scheduling strategies
- Goal:
 - reduce complexity while maintaining quality

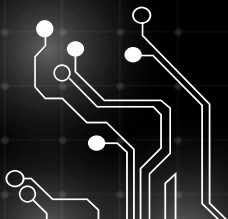
Progress This Week

1. Explored MAPF libraries
2. Selected a framework (Python + visualization)
 - a. Chosen because of:
 - i. Ease of implementation
 - ii. Strong visuals
 - iii. Premade environments
 - iv. Already implemented SIPP and CBS
3. We implemented and tested:
 - a. MA-CBS
 - b. SAT solver
 - c. Incremental Coordination



Next Steps

1. Run all algorithms across environments
 - a. Generate new environments representing bottlenecks, narrow passages, open spaces, etc.
2. Collect metrics:
 - a. runtime
 - b. solution cost
 - c. makespan
 - d. number of conflicts resolved
3. Compare performance
4. Use results to begin building selection model





Thanks!

Questions?

CREDITS: This presentation template was created by [**Slidesgo**](#), and includes icons by [**Flaticon**](#) and infographics & images by [**Freepik**](#)

